Przekazywanie obiektów jako parametrów funkcji

Lekcja 16 - Programowanie aplikacji

Klasa 4 technikum - technik programista - INF.04

```
Pracownik p;
pobierzDane(p);
wyswietlDane(p);

Pracownik* w = new Pracownik();
pobierzDane(w);
wyswietlDane(w);
delete w;
```

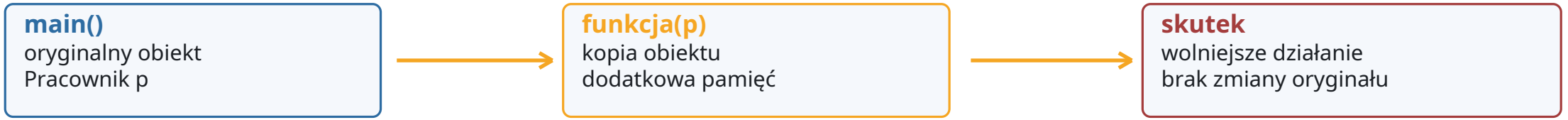
Cele lekcji

- porównasz przekazywanie obiektu przez wartość, referencję i wskaźnik;
- zastosujesz parametr const w funkcji, która tylko odczytuje dane obiektu;
- zdefiniujesz funkcję zwracającą referencję lub wskaźnik do obiektu;
- zapiszesz wywołania funkcji dla obiektu statycznego i dynamicznego;
- unikniesz typowych błędów: kopiowania dużych obiektów, braku delete, zmiany obiektu const.

Efekt kształcenia
INF.04.4 - programowanie obiektowe: definiowanie klas i metod oraz stosowanie zasad programowania obiektowego w programach.

Dlaczego nie zawsze przez wartość?

Obiekty mogą przechowywać wiele pól, tablice, inne obiekty albo dane zaalokowane dynamicznie. Przekazanie przez wartość tworzy kopię.



- referencja i wskaźnik przekazują dostęp do tego samego obiektu;
- const chroni obiekt przed przypadkową zmianą w funkcji odczytującej dane.

Trzy sposoby przekazania obiektu

Sposób	Przykład	Kopia?	Zmiana oryginału?	Typowe użycie
przez wartość	void f(Pracownik p)	tak	nie	małe obiekty, gdy potrzebna kopia
przez referencję	void f(Pracownik& p)	nie	tak	modyfikacja obiektu
przez stałą referencję	void f(const Pracownik& p)	nie	nie	bezpieczny odczyt danych
przez wskaźnik	void f(Pracownik* p)	nie	tak	obiekty dynamiczne, możliwość nullptr

Parametr wejściowy: const referencja

Gdy funkcja ma tylko korzystać z danych obiektu, najlepiej przekazać go przez stałą referencję.

odczyt bez kopiowania

```
void wyswietlDane(const Pracownik& p) {
    cout << "Imie: " << p.imie << endl;
    cout << "Nazwisko: " << p.nazwisko << endl;

    // p.imie = "Jan";  // błąd: const
}
```

Zapamiętaj

const Pracownik& oznacza: funkcja dostaje dostęp do oryginalnego obiektu, ale nie może zmienić jego pól. To dobry wybór dla parametrów wejściowych.

Parametr wyjściowy: referencja

Funkcja może pobrać dane i zmienić przekazany obiekt. Wtedy parametr nie powinien być const.

modyfikacja obiektu

```
Pracownik& pobierzDane(Pracownik& p) {
    cout << "Imie = ";
    cin >> p.imie;
    cout << "Nazwisko = ";
    cin >> p.nazwisko;
    return p;
}

Pracownik pracownik;
pobierzDane(pracownik);
```

Co się dzieje?

p jest drugim sposobem dostępu do tego samego obiektu, który istnieje w main(). Dane wpisane w funkcji zostają w obiekcie pracownik.

Wariant wskaźnikowy

Wskaźnik przechowuje adres obiektu. Do pól obiektu wskazywanego przez wskaźnik odwołujemy się operatorem strzałkowym ->.

obiekt dynamiczny na stercie

```
Pracownik* w = new Pracownik();

w->imie = "Jan";
w->nazwisko = "Kowalski";

wyswietlDane(w);

delete w;
```

Ważne

new tworzy obiekt dynamiczny, a delete zwalnia pamięć. Brak delete może prowadzić do wycieku pamięci.

Funkcje z parametrami wskaźnikowymi

przekazywanie wskaźnika do obiektu

```
Pracownik* pobierzDane(Pracownik* w_p) {
    cout << "Imie = ";
    cin >> w_p->imie;
    cout << "Nazwisko = ";
    cin >> w_p->nazwisko;
    return w_p;
}

void wyswietlDane(const Pracownik* w_p) {
    if (w_p != nullptr) {
        cout << "Imie: " << w_p->imie << endl;
        cout << "Nazwisko: " << w_p->nazwisko << endl;
    }
}
```

Referencja czy wskaźnik?

Referencja
Użyj, gdy obiekt musi istnieć i funkcja ma pracować na tym konkretnym obiekcie. Składnia jest prosta: p.imie.

Wskaźnik
Użyj, gdy pracujesz z obiektem dynamicznym albo gdy dopuszczasz brak obiektu. Warto sprawdzić nullptr. Składnia: w->imie.

- dla zwykłej modyfikacji najczęściej wystarczy referencja;
- dla odczytu dużego obiektu stosuj const referencję;
- dla obiektów tworzonych przez new pamiętaj o delete.

Ćwiczenie lekcyjne

Zdefiniuj klasę Uczen z polami: imie, nazwisko, klasa, grupa, numer_w_dzienniku oraz strukturą DataUrodzenia.

- funkcja pobierzDane(Uczen&) ma wczytać dane ucznia;
- funkcja wyswietlDane(const Uczen&) ma wyświetlić dane bez kopiowania obiektu;
- następnie przygotuj wariant z obiektem dynamicznym i wskaźnikiem Uczen*.

punkt startowy

```
struct DataUrodzenia { int dzien, miesiac, rok; };
class Uczen { public: string imie, nazwisko, klasa, grupa; int numer; DataUrodzenia data; };
```

Najczęstsze błędy

- brak const przy funkcji, która tylko wyświetla dane;
- niepotrzebne przekazywanie dużego obiektu przez wartość;
- użycie kropki zamiast operatora -> przy wskaźniku;
- brak sprawdzenia nullptr, gdy wskaźnik może nie wskazywać obiektu;
- brak delete po obiekcie utworzonym przez new.

Podsumowanie

- Przekazanie przez wartość tworzy kopię obiektu.
- Referencja pozwala pracować na oryginalnym obiekcie bez kopiowania.
- const referencja jest najlepsza do bezpiecznego odczytu dużych obiektów.
- Wskaźnik przechowuje adres obiektu i wymaga operatora ->.
- Obiekty dynamiczne utworzone przez new trzeba zwalniać przez delete.